

Markupp

Servidor open-source self-hosted de anotações em Markdown

Renato Freitas Nicolás Arthur Nicolás Pitz Gabriela Riedel

Instituto Federal de Santa Catarina

Maio de 2026



INSTITUTO FEDERAL
Santa Catarina
Campus São José

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Contexto e problema

Contexto e problema

- Anotações em *Markdown* e saídas geradas por agentes de IA crescem rapidamente em volume.
- Construir um *segundo cérebro* digital é uma prática consolidada em *Personal Knowledge Management*.
- Soberania do dado: o acervo permanece sob controle direto do usuário, em servidor próprio (*self-hosted*).
- Interface aberta a múltiplos clientes, humanos e agentes, sem amarrar o conteúdo a um aplicativo único.
- Desempenho e escalabilidade internos para acompanhar o crescimento da base.

Objetivos

Objetivos

Objetivo geral

- Desenvolver um servidor *self-hosted* que centralize arquivos em *Markdown* e exponha interface aberta de acesso a usuários e a agentes de IA.

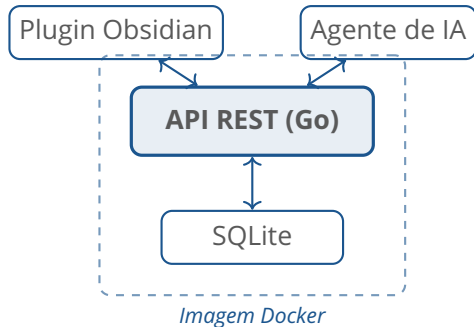
Objetivos específicos

- Implementar operações de criação, leitura, edição, exclusão e renomeação de notas no servidor.
- Oferecer plugin do Obsidian como cliente humano de referência.
- Expor uma API REST como única interface de acesso ao servidor.

Arquitetura

Arquitetura

- Cliente humano (Obsidian) e cliente de software (agente de IA) consomem a mesma interface REST.
- O servidor encapsula a persistência: o cliente nunca acessa o armazenamento diretamente.
- O sistema é distribuído como uma imagem Docker que sobe o servidor e o banco em conjunto.

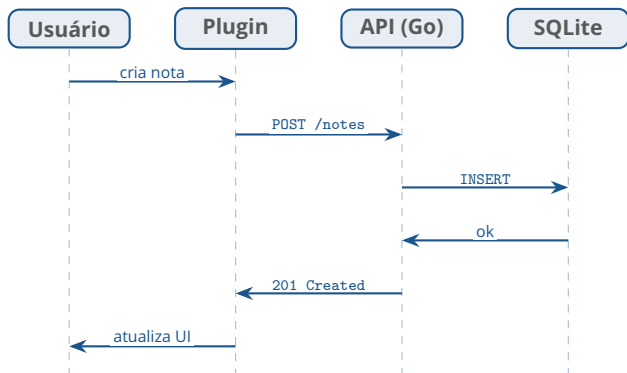


Fluxo de uma operação

Fluxo de uma operação

Criação de uma nota:

- 1 Usuário aciona “criar nota” no Obsidian.
- 2 Plugin envia `POST /notes` ao servidor.
- 3 API valida o *payload* e delega ao repositório.
- 4 Repositório grava conteúdo e metadados em uma única transação SQLite.
- 5 Servidor responde 201 Created; plugin atualiza a interface.



Decisões técnicas

Decisões técnicas

- **Servidor em Go:** alto desempenho e baixo consumo de recursos, mantendo o sistema leve para *self-hosting* em hardware modesto.
- **SQLite como armazenamento:** zero configuração; conteúdo, anexos e metadados gravados em uma única transação; *backup* reduzido a copiar um arquivo.
- **API REST:** contrato HTTP documentado que permite a qualquer cliente, humano ou agente de IA, integrar-se ao sistema.
- **Plugin do Obsidian em TypeScript:** cliente humano de referência; o servidor permanece aberto a outros clientes.
- **Distribuição via Docker:** imagem reprodutível que sobe servidor e banco em conjunto, reduzindo o atrito de instalação.

Estado atual

Estado atual

- CRUD de notas operacional, atravessando plugin do Obsidian, API REST e SQLite.
- Plugin do Obsidian sincronizado com o servidor.
- Contrato da API descrito em OpenAPI.
- Distribuição via *compose* para execução local.
- *Release* v0.2.0 publicada (Sprint 2).

Trabalhos futuros e limitações

Trabalhos futuros e limitações

Próximas etapas

- Busca léxica em títulos e em conteúdo.
- Indexação vetorial e busca semântica.
- Integração com agentes de IA via MCP (Anthropic, 2024).
- Taxonomias e classificação de notas.

Limitações deliberadas do MVP

- Ausência de TLS, autenticação e *rate limit*.
- Demonstração restrita a ambiente de rede interna.
- Camadas de segurança serão tratadas antes de qualquer exposição pública do servidor.

Referências I



ANTHROPIC. **Model Context Protocol: Specification and Documentation**. 2024. Disponível em: <https://modelcontextprotocol.io/>.